



Universidad
Nacional
de Córdoba



Facultad de Matemática,
Astronomía, Física y
Computación

Formulación de un sudoku mediante programación lineal entera binaria y su implementación en Julia y Python

Bocco, Noelia - Ruiz, Luciana

7 de junio de 2026

Resumen

En este trabajo se aborda el modelado y la resolución del Sudoku de tamaño 9×9 mediante técnicas de programación lineal entera. Para ello, el juego se formula como un problema de Programación Lineal Entera Binaria (BILP), donde las reglas de unicidad en celdas, filas, columnas y subcuadrículas se representan mediante un conjunto de restricciones lineales sobre variables binarias.

El modelo fue implementado utilizando dos herramientas de código abierto: Julia, mediante la interfaz JuMP, y Python, a través de la biblioteca PuLP. Los experimentos realizados permitieron comparar el desempeño y los tiempos de ejecución de ambas implementaciones frente a instancias de distinta dificultad.

La implementación en Julia presentó menores tiempos promedio en la mayoría de las instancias evaluadas. Además, se observó un comportamiento interesante: la dificultad percibida por una persona al resolver un Sudoku no necesariamente coincide con la dificultad computacional del problema. En particular, varias instancias con menor cantidad de pistas iniciales requirieron tiempos de resolución iguales o incluso menores que algunas instancias consideradas más sencillas.

1. Introducción

El Sudoku es un popular juego de lógica y naturaleza combinatoria. En su versión estándar de 9×9 , consiste en una cuadrícula de 81 casillas dividida en nueve subcuadrículas de 3×3 . A partir de algunos números ya ubicados en el tablero, denominados pistas (clues), el objetivo es completar las celdas restantes utilizando los dígitos del 1 al 9 sin repetir ningún valor en una misma fila, columna o subcuadrícula.

La estructura del Sudoku permite modelarlo como un problema de Programación Lineal Entera Binaria (BILP). En este enfoque, cada posible asignación de un número a una celda se representa mediante una variable binaria que indica si dicha asignación forma parte o no de la solución. A diferencia de otros problemas de optimización, el Sudoku no busca maximizar ni minimizar una función objetivo, sino encontrar una solución factible que satisfaga todas las restricciones impuestas por las reglas del juego.

Más allá de su carácter recreativo, este tipo de modelado aparece en numerosos problemas de optimización, como la planificación de horarios, la asignación de recursos, la programación de tareas y el diseño de redes.

Como base teórica para este trabajo se tomó el artículo *Formulation of Sudoku Puzzle Using Binary Integer Linear Programming and Its Implementation in Julia, Python, and Minizinc* de Bukhari et al. (2022), donde se presenta una formulación del problema mediante variables indexadas por fila, columna y valor.

A partir de dicha formulación, se desarrollaron implementaciones en Julia utilizando JuMP y en Python utilizando PuLP. El objetivo principal consiste en comparar el rendimiento de ambas herramientas frente a tableros de distinta dificultad, analizando sus tiempos de resolución y el comportamiento de los solucionadores a medida que varía la cantidad de pistas iniciales disponibles.

Para ello, se consideraron cuatro niveles de dificultad, definidos según el número de pistas presentes en el tablero y la cantidad mínima de pistas por fila y columna, tal como se muestra en el Cuadro 1.

Cuadro 1: Niveles de dificultad en los tableros de Sudoku analizados.

Nivel de Dificultad	Número de Pistas	Mínimo de Pistas por Fila/Columna
Fácil (Easy)	36 – 46	4
Moderado (Moderate)	32 – 35	3
Difícil (Hard)	28 – 31	2
Extremadamente Difícil	17 – 27	0

2. Trabajo realizado: El Modelo Estándar

En esta sección se presenta la formulación matemática del modelo de Programación Lineal Entera Binaria (BILP) utilizado para resolver el Sudoku clásico de tamaño 9×9 .

2.1. Formulación del Problema BILP

El modelo se construye a partir de un conjunto de índices, variables de decisión y restricciones que representan las reglas del juego.

2.1.1. Índices y parámetros del modelo

Para describir el problema se utilizan los siguientes índices:

- $i \in \{1, 2, \dots, 9\}$: Representa el índice de la fila dentro del tablero de Sudoku.
- $j \in \{1, 2, \dots, 9\}$: Representa el índice de la columna dentro del tablero de Sudoku.
- $k \in \{1, 2, \dots, 9\}$: Representa el valor numérico o dígito asignado a la coordenada (i, j) .

Además, se define el conjunto Ω , que contiene las pistas iniciales del Sudoku. Cada elemento de este conjunto representa una celda cuyo valor es conocido de antemano y debe mantenerse fijo durante la resolución del problema.

2.1.2. Variables de Decisión

La asignación de un número a una posición específica del tablero se representa mediante la variable binaria.

$$x_{ijk} = \begin{cases} 1 & \text{si la celda en la fila } i \text{ y columna } j \text{ contiene el número } k, \\ 0 & \text{en caso contrario.} \end{cases} \quad (1)$$

De esta manera, el estado completo del Sudoku queda determinado por el conjunto de variables que toman valor 1.

2.1.3. Función Objetivo

El Sudoku constituye un problema de factibilidad, ya que el objetivo no es optimizar una cantidad determinada, sino encontrar una asignación de valores que satisfaga todas las restricciones del modelo. Por esta razón, se utiliza una función objetivo trivial:

$$\text{mín } Z = \sum_{i=1}^9 \sum_{j=1}^9 \sum_{k=1}^9 0 \cdot x_{ijk} \quad (2)$$

Observación. El modelo utiliza $9^3 = 729$ variables binarias, una por cada posible asignación de un dígito a una celda del tablero. Las restricciones lineales definidas garantizan que toda solución factible corresponda a un Sudoku válido.

2.2. Restricciones de Unicidad

Las reglas del Sudoku se modelan mediante un conjunto de restricciones lineales que garantizan la validez de la solución obtenida:

1. **Asignación única por celda:** Cada posición (i, j) debe contener un único dígito k .

$$\sum_{k=1}^9 x_{ijk} = 1, \quad \forall i, j \in \{1, \dots, 9\} \quad (3)$$

2. **Unicidad en filas:** Cada número k debe aparecer exactamente una vez en cada fila i .

$$\sum_{j=1}^9 x_{ijk} = 1, \quad \forall i, k \in \{1, \dots, 9\} \quad (4)$$

3. **Unicidad en columnas:** Cada número k debe figurar una sola vez en cada columna j .

$$\sum_{i=1}^9 x_{ijk} = 1, \quad \forall j, k \in \{1, \dots, 9\} \quad (5)$$

4. **Unicidad en subcuadrículas de 3×3 :** Para asegurar que ningún dígito k se repita en los 9 bloques menores, definimos los índices de inicio $p, q \in \{1, 2, 3\}$:

$$\sum_{i=3p-2}^{3p} \sum_{j=3q-2}^{3q} x_{ijk} = 1, \quad \forall k \in \{1, \dots, 9\}, \quad \forall p, q \in \{1, 2, 3\} \quad (6)$$

5. **Pistas iniciales (*Clues*):** Las celdas previamente definidas por el tablero se mantienen fijas durante todo el proceso de resolución:

$$x_{ijk} = 1, \quad \forall (i, j, k) \in \Omega \quad (7)$$

			2	4				1
							6	
						5	7	4
		3		8			1	
5		4						8
			7					
			6		9			
		8				6		
	7				4		9	2

Figura 1: Instancia de un Sudoku de 9×9 con 22 pistas dadas.

2.3. Ilustración Práctica de las Restricciones

La Figura 1 muestra una instancia de Sudoku de 9×9 con 22 pistas iniciales. Estas pistas se incorporan al modelo mediante restricciones de fijación de variables. Por ejemplo, el valor 2 ubicado en la primera fila y cuarta columna se representa como

$$x_{142} = 1.$$

De manera análoga, cada una de las pistas del tablero genera una restricción del mismo tipo, reduciendo el espacio de búsqueda que debe explorar el solver.

$$\begin{aligned} x_{142} &= x_{154} = x_{191} = x_{286} = x_{375} = x_{387} = x_{394} = 1, \\ x_{433} &= x_{458} = x_{471} = x_{515} = x_{534} = x_{598} = x_{647} = 1, \\ x_{746} &= x_{769} = x_{838} = x_{876} = x_{927} = x_{964} = x_{989} = x_{992} = 1. \end{aligned}$$

A partir de estas asignaciones iniciales, las restricciones de unicidad en filas, columnas y subcuadrículas garantizan que ningún dígito pueda repetirse. Por ejemplo, en la subcuadrícula superior derecha ya se encuentran fijados los valores 1, 4, 5, 6 y 7, por lo que las celdas restantes de dicho bloque solo podrán tomar valores compatibles con las restricciones del modelo. De esta forma, la interacción entre las pistas iniciales y las restricciones de unicidad permite reducir progresivamente el conjunto de soluciones factibles hasta obtener una solución válida del Sudoku.

2.4. Implementación Computacional

2.4.1. Modelado en Julia con JuMP

Julia es un lenguaje de programación de alto rendimiento ampliamente utilizado en computación científica. En este trabajo se empleó JuMP, una biblioteca que permite modelar y resolver problemas de optimización mediante una sintaxis simple y cercana a su formulación matemática.

Para visualizar el código completo, el entorno de trabajo y los experimentos realizados, puede accederse al siguiente enlace: [Julia](#)

2.4.2. Modelado en Python con PuLP

Python es un lenguaje de programación muy utilizado por su simplicidad, legibilidad y gran cantidad de bibliotecas disponibles. En este trabajo se utilizó la biblioteca PuLP, que permite modelar y resolver problemas de programación lineal de manera sencilla dentro de Python.

El código completo y los experimentos realizados pueden consultarse en el siguiente enlace: [Python](#)

2.5. Diseño Experimental y Banco de Pruebas

Para validar el rendimiento de este modelo BILP, se recopilaron y evaluaron 10 instancias de prueba para cada uno de los cuatro niveles de dificultad definidos: fácil, moderado, difícil y extremadamente difícil.

Para evitar que factores externos del procesador alteraran las mediciones, cada caso de prueba se ejecutó diez veces consecutivas bajo idénticas condiciones. En lugar de registrar un único intento, calculamos el promedio de estas diez pasadas; de esta manera, se redujo el impacto de cualquier fluctuación en la velocidad de la computadora, logrando una base de datos uniforme y confiable para comparar el rendimiento de Julia y Python de forma justa.

3. Extensiones del Modelo: Variantes del Juego

Una de las grandes ventajas de formular el Sudoku a través de la Programación Lineal Entera Binaria es la **flexibilidad del modelo**. Si las reglas lógicas cambian o se añaden nuevas condiciones, no es necesario reescribir el sistema desde cero; basta con incorporar nuevas familias de restricciones matemáticas al conjunto original.

A continuación, se presentan dos variantes del juego y cómo se modelan formalmente.

3.1. Variante 1: Sudoku Diagonal (Sudoku-X)

En el Sudoku Diagonal, además de cumplir las reglas tradicionales de filas, columnas y bloques, los números del 1 al 9 deben aparecer exactamente una vez en las dos grandes diagonales principales del tablero.

A nivel algebraico, esto requiere agregar únicamente dos nuevas restricciones al modelo original, barriendo los índices donde la fila coincide con la columna ($i = j$) y donde son opuestas ($i + j = 10$):

1. **Unicidad en la Diagonal Principal (de izquierda a derecha):**

$$\sum_{i=1}^9 x_{i,i,k} = 1, \quad \forall k \in \{1, \dots, 9\} \quad (8)$$

2. **Unicidad en la Diagonal Secundaria (de derecha a izquierda):**

$$\sum_{i=1}^9 x_{i,10-i,k} = 1, \quad \forall k \in \{1, \dots, 9\} \quad (9)$$

	2	1			4	8	3	
4		3	5		8	2		1
	6		1	3				
	9	2		7		5	4	
1		5	3					
		7						8
	5			2		3		
2	3						8	6
	1		6	8	3	4	5	

3.2. Variante 2: Sudoku Killer

Además de las restricciones tradicionales (filas, columnas y bloques 3×3 , el tablero se divide en regiones llamadas jaulas. Cada jaula tiene:

- un conjunto de celdas
- un número objetivo (la suma)
- los números dentro de la jaula no pueden repetirse

Formulación Matemática: Sea C_m la jaula m y S_m su suma objetivo.

1. Restricción de suma dentro de la jaula:

$$\sum_{(i,j) \in C_m} \sum_{k=1}^9 kx_{ijk} = S_m$$

para cada jaula m

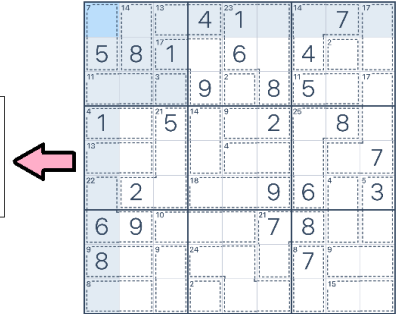
2. Restricción de unicidad en la jaula: Para cada C_m y para cada dígito k

$$\sum_{(i,j) \in C_m} x_{ijk} \leq 1$$

Por ejemplo, consideremos la jaula formada por las celdas (4, 3), (5, 3), (6, 2) y (6, 3).

- **Restricción de suma de la jaula:**
objetivo es 21,

$$\sum_{k=1}^9 kx_{4,3,k} + \sum_{k=1}^9 kx_{5,3,k} + \sum_{k=1}^9 kx_{6,2,k} + \sum_{k=1}^9 kx_{6,3,k} = 21$$



- **Restricción de unicidad en la jaula:**

$$x_{4,3,k} + x_{5,3,k} + x_{6,2,k} + x_{6,3,k} \leq 1, \forall k \in 1, \dots, 9$$

4. Resultados y Discusión

En esta sección se presentan y analizan los resultados obtenidos a partir de las distintas pruebas realizadas. Los datos recolectados permiten comparar el rendimiento de las implementaciones desarrolladas en Python y Julia, así como estudiar el comportamiento de los solucionadores frente a instancias de Sudoku de diferente dificultad.

4.1. Eficiencia Comparativa entre Lenguajes

La Figura 2 muestra los tiempos de cómputo obtenidos para las implementaciones desarrolladas en Python (PuLP) y Julia (JuMP) sobre instancias de Sudoku con distintos niveles de dificultad. Para cada nivel se realizaron diez ejecuciones independientes, representadas mediante líneas punteadas, mientras que las líneas sólidas indican el tiempo promedio correspondiente.

Se observa que la implementación en Julia presenta tiempos de ejecución considerablemente menores que la implementación en Python en todas las categorías analizadas. Mientras que Python registra tiempos promedio cercanos a 0.10–0.14 segundos, Julia mantiene valores alrededor de 0.02–0.05 segundos, evidenciando una ventaja computacional significativa.

Otro aspecto relevante es la baja dispersión observada en los tiempos de Julia para la mayoría de las instancias. Esto sugiere un comportamiento más estable y consistente frente a variaciones en las configuraciones iniciales del tablero. En contraste, Python exhibe una mayor variabilidad entre ejecuciones, particularmente en los niveles Fácil y Moderado.

En conjunto, los resultados obtenidos permiten concluir que la implementación desarrollada en Julia mediante JuMP ofrece un mejor desempeño computacional que la implementación realizada en Python con PuLP, tanto en términos de velocidad de resolución como de estabilidad entre ejecuciones.

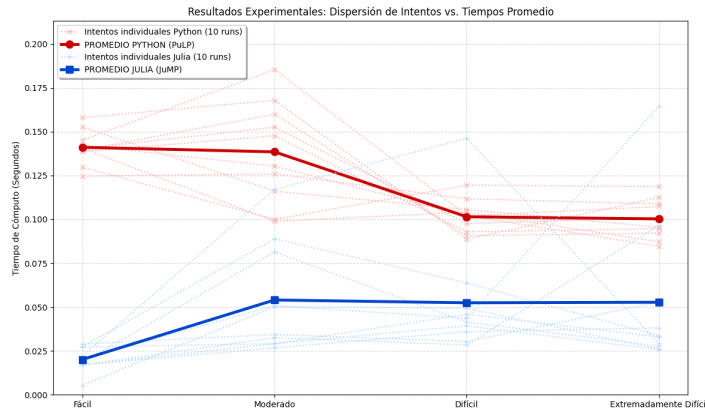


Figura 2: Tiempo de cada intento vs tiempo promedio según Python y Julia

4.2. El Comportamiento Contraintuitivo de los Solvers

Más allá de las diferencias observadas entre Julia y Python, los resultados revelaron un comportamiento interesante relacionado con la dificultad percibida del Sudoku. Como puede apreciarse en la Figura 2, los tiempos de resolución no aumentan necesariamente a medida que el Sudoku se vuelve más difícil desde la perspectiva de un jugador humano.

De hecho, las instancias clasificadas como difíciles y extremadamente difíciles presentaron tiempos de ejecución similares o incluso menores que algunas instancias fáciles. Este resultado puede parecer contraintuitivo, ya que una persona suele asociar una menor cantidad de pistas iniciales con una mayor dificultad de resolución.

Este comportamiento plantea una pregunta : ¿por qué algunas instancias aparentemente más difíciles pueden resolverse en menos tiempo? Una posible explicación surge al analizar la geometría del espacio de soluciones inducido por las restricciones del modelo.

4.3. Interpretación Geométrica

Una forma intuitiva de comprender este comportamiento es analizar el espacio de soluciones del modelo. En programación lineal entera, las restricciones definen una región factible que puede interpretarse geométricamente como un politopo de alta dimensión.

Las distintas configuraciones iniciales del Sudoku modifican la estructura de esta región factible. Dependiendo de cómo interactúan las restricciones impuestas por las pistas iniciales, el solver puede encontrar con mayor o menor facilidad una solución factible. Por este motivo, una instancia que resulta sencilla para una persona no necesariamente es la más rápida de resolver para un algoritmo de optimización.

En particular, se observó que algunas instancias con menos pistas permitieron tiempos de resolución menores, lo que sugiere que la complejidad computacional está más relacionada con la estructura matemática generada por las restricciones que con la dificultad percibida del tablero.

4.4. ¿Qué pasa con un tablero 100 % vacío?

En este caso, el solver debe construir una solución desde cero, sin información inicial que reduzca el espacio de búsqueda.

Esto sugiere que existe un equilibrio entre la cantidad de pistas y el rendimiento computacional: muy pocas restricciones generan un espacio de búsqueda más amplio, mientras que demasiadas pueden aumentar la complejidad del modelo. En nuestros experimentos, las instancias difíciles y extremadamente difíciles presentaron los menores tiempos de resolución.

Además, mientras que Julia obtuvo mejores tiempos en la mayoría de los casos, en el tablero vacío Python mostró un tiempo promedio menor, evidenciando diferencias en el comportamiento de ambas implementaciones frente a este caso extremo.

5. Conclusiones

En este trabajo se formuló el problema del Sudoku clásico de (9×9) como un modelo de Programación Lineal Entera Binaria. La formulación permitió representar las reglas del juego mediante restricciones lineales de unicidad sobre celdas, filas, columnas y subcuadrículas, junto con restricciones adicionales para fijar las pistas iniciales.

Las implementaciones desarrolladas en Python con PuLP y en Julia con JuMP permitieron comparar el desempeño computacional de ambas herramientas frente a instancias de distinta dificultad. Julia presentó menores tiempos promedio en todas las instancias evaluadas. Aunque se observaron algunos valores atípicos, en general mostró un comportamiento estable y competitivo.

Finalmente, se observó que la dificultad percibida por una persona no necesariamente coincide con la dificultad computacional del modelo. Esto muestra que, en problemas combinatorios como el Sudoku, la cantidad de pistas no es el único factor relevante: también influye la forma en que dichas pistas restringen el espacio factible.

Referencias

- [1] Ming S. Hung. A Polynomial Simplex Method for the Assignment Problem. Operations Research, 31(3):595–600, 1983.